FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

# CS0065
# (INTELLIGENT SYSTEMS)

## TECHNICAL ASSESSMENT

# 3

## CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

| | |
|---|---|
| **Student Name** | Fritz Mardaniel L. Sablan |
| **Section:** | AN42 |
| **Professor:** | Ms. Crisola G. Tan |
| **Date:** | 05/10/2026 |

## I. BACKGROUND INFORMATION

**Case-Based Reasoning (CBR)** is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

## II. TECHNICAL ASSESSMENT

### CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

***Objective***

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

**Materials Needed**

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

**General Instructions**

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   *(Example: TA2_Patilano_Loverez.pdf)*
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

**1. Form a pair**

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

**2. Choose ONE (1) case study**

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

**Case Study Options:**

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

**3. Define your case base**

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

**4. Implement similarity assessment**

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

**5. Adapt the solution**

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

**6. Demonstrate learning (retain stage)**

After generating a final solution, store the new case into your case base.

**Your program must show:**

✓ "New case saved" or similar confirmation

✓ Updated case list

**7. Display clear program outputs**

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

**8. Submission Requirements**

Submit the following:

**A. Program File**

- Python

**B. Short Documentation (1–2 pages)**

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

**C. Pair Reflection (5–7 sentences)**

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

**Case Studies:**

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

**Learning Outcome:**

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

**Scenario:**

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

**Learning:**
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

| Case Study 3: Intelligent Tutoring System |
|---|
| A CBR system identifies the learning struggles of students.<br>Student submits a math solution with repeated errors.<br>System retrieves past students with similar error patterns (e.g., misusing formulas).<br>It adapts feedback and personalized lessons based on successful interventions.<br>**Learning:**<br>System improves by retaining new student profiles and outcomes. |

**Your Code - Laboratory Tasks** ⇩

Case Study 1: Disaster Response Decision Support System

The case base contains five previous disaster cases that serve as examples for solving new disaster-response problems. Each case contains two main parts, the problem attributes and the solution attributes. For the new case, the system considers an urban typhoon with expected flooding, a population of 350,000, flood risk of 5, severity of 4, and 24 hours of lead time. The case base is then used to identify which previous disaster is most applicable.

The system uses a weighted similarity method to compare the new disaster with every case in the case base. The weights determine how important each attribute is when deciding whether two disaster cases are similar. Disaster type has the highest weight at 25%, while lead time has the lowest at 10%.

Adaptation Logic

a. Population-based resource scaling

- The number of evacuees, relief packs, and medical teams is adjusted according to the ratio between the new population and the population of the retrieved case.

b. Rescue boat adjustment

- Rescue boats are adjusted using both population size and flood risk. This means a larger population and greater flood risk result in more rescue boats being recommended.

c. Severity adjustment

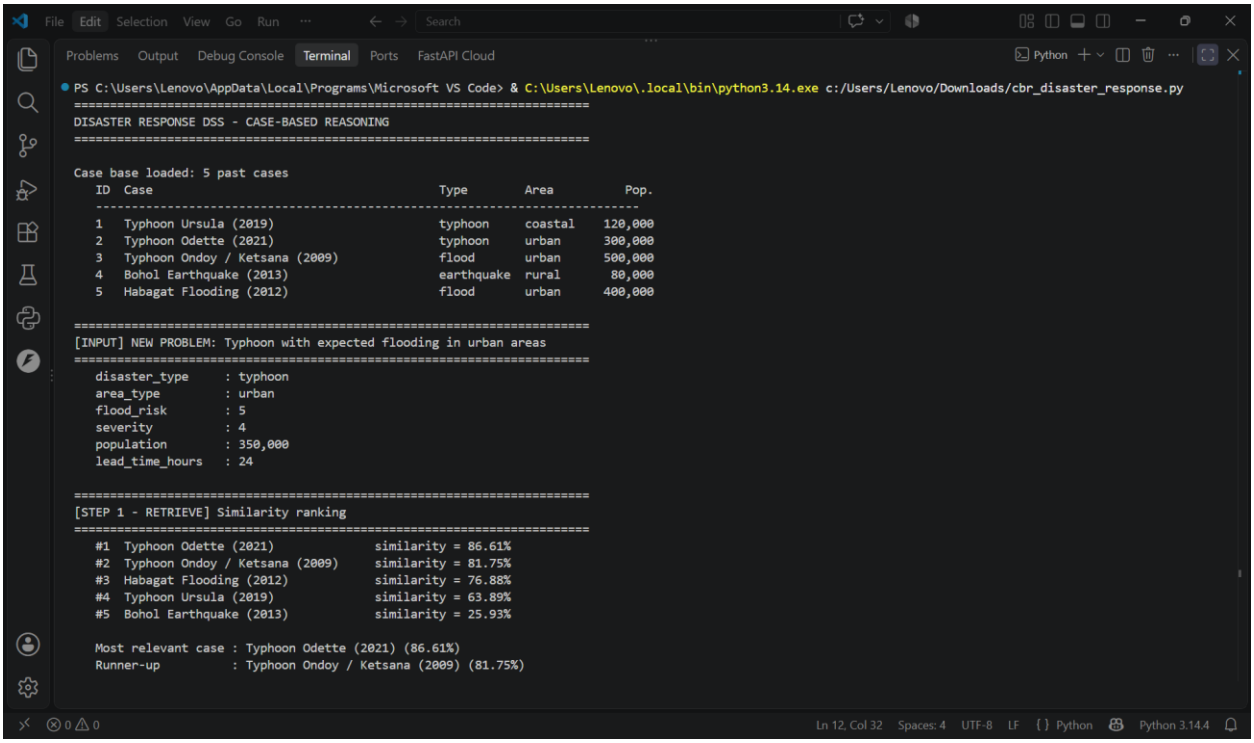
- If the new disaster has a higher severity than the retrieved case, the system increases the number of planned evacuees by 10% for each additional severity level.

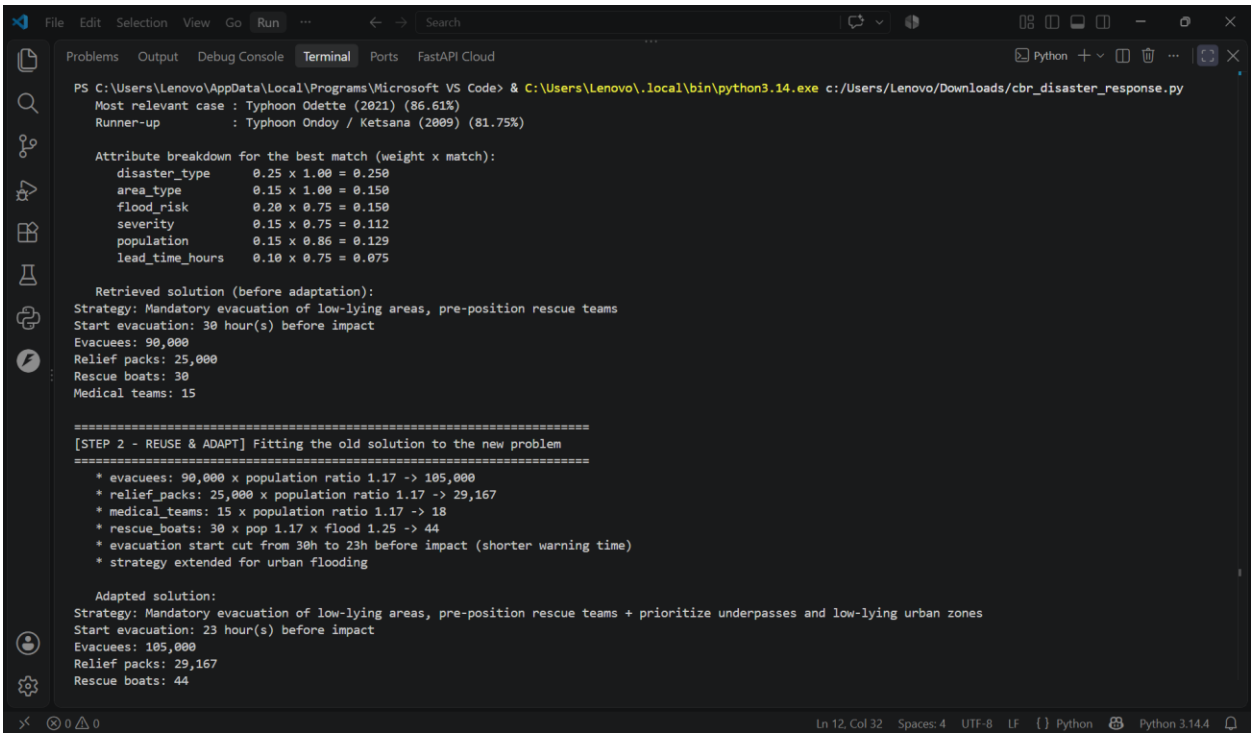
d. Lead-time adjustment

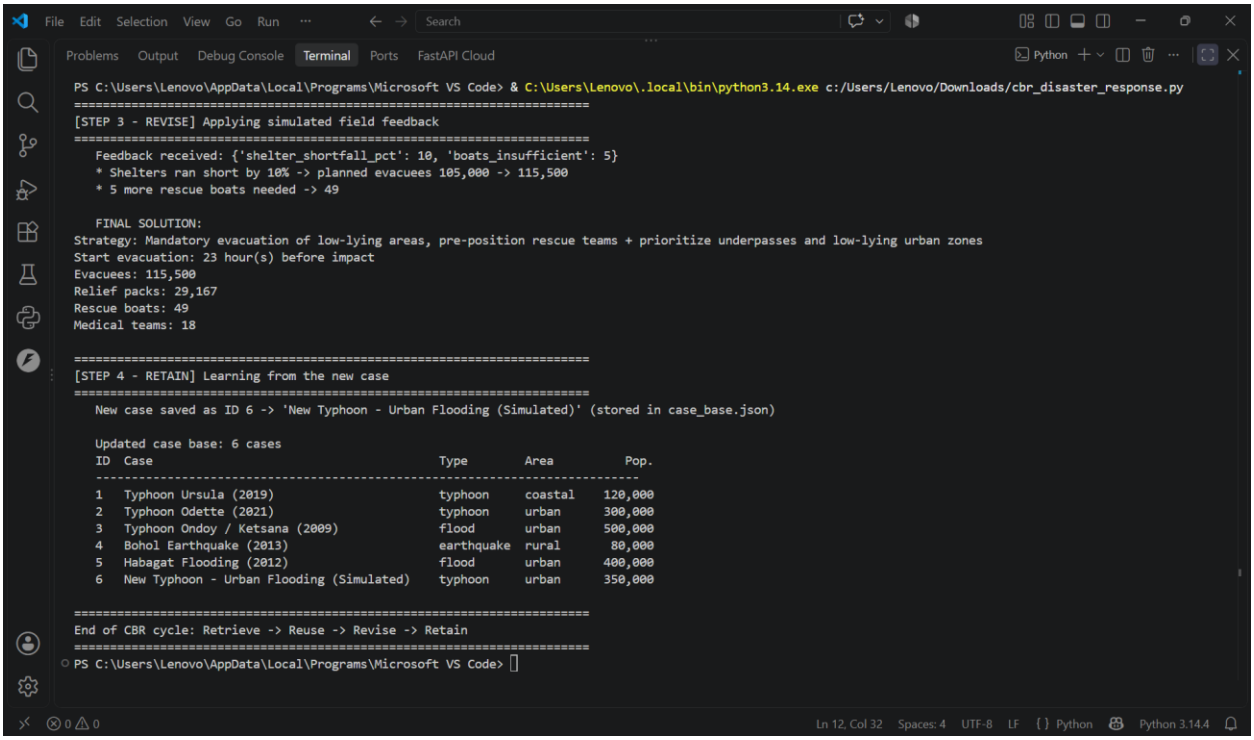
- If the new disaster has less warning time than the previous case's evacuation schedule, the system changes the evacuation start time to fit within the available warning period.

e. Urban flooding adjustment

- If the new situation is an urban area with flood risk of 4 or higher, the evacuation strategy is expanded to specifically prioritize underpasses and low-lying urban zones.


PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Lenovo\.local\bin\python3.14.exe c:/Users/Lenovo/Downloads/cbr_disaster_response.py
DISASTER RESPONSE DSS - CASE-BASED REASONING
Case base loaded: 5 past cases
ID Case Type Area Pop.
1 Typhoon Ursula (2019) typhoon coastal 120,000
2 Typhoon Odette (2021) typhoon urban 300,000
3 Typhoon Ondoy / Ketsana (2009) flood urban 500,000
4 Bohol Earthquake (2013) earthquake rural 80,000
5 Habagat Flooding (2012) flood urban 400,000
[INPUT] NEW PROBLEM: Typhoon with expected flooding in urban areas
disaster_type : typhoon
area_type : urban
flood_risk : 5
severity : 4
population : 350,000
lead_time_hours : 24
[STEP 1 - RETRIEVE] Similarity ranking
#1 Typhoon Odette (2021) similarity = 86.61%
#2 Typhoon Ondoy / Ketsana (2009) similarity = 81.75%
#3 Habagat Flooding (2012) similarity = 76.88%
#4 Typhoon Ursula (2019) similarity = 63.89%
#5 Bohol Earthquake (2013) similarity = 25.93%
Most relevant case : Typhoon Odette (2021) (86.61%)
Runner-up : Typhoon Ondoy / Ketsana (2009) (81.75%)


PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Lenovo\.local\bin\python3.14.exe c:/Users/Lenovo/Downloads/cbr_disaster_response.py
Most relevant case : Typhoon Odette (2021) (86.61%)
Runner-up : Typhoon Ondoy / Ketsana (2009) (81.75%)
Attribute breakdown for the best match (weight x match):
disaster_type 0.25 x 1.00 = 0.250
area_type 0.15 x 1.00 = 0.150
flood_risk 0.20 x 0.75 = 0.150
severity 0.15 x 0.75 = 0.112
population 0.15 x 0.86 = 0.129
lead_time_hours 0.10 x 0.75 = 0.075
Retrieved solution (before adaptation):
Strategy: Mandatory evacuation of low-lying areas, pre-position rescue teams
Start evacuation: 30 hour(s) before impact
Evacuees: 90,000
Relief packs: 25,000
Rescue boats: 30
Medical teams: 15
[STEP 2 - REUSE & ADAPT] Fitting the old solution to the new problem
* evacuees: 90,000 x population ratio 1.17 -> 105,000
* relief_packs: 25,000 x population ratio 1.17 -> 29,167
* medical_teams: 15 x population ratio 1.17 -> 18
* rescue_boats: 30 x pop 1.17 x flood 1.25 -> 44
* evacuation start cut from 30h to 23h before impact (shorter warning time)
* strategy extended for urban flooding
Adapted solution:
Strategy: Mandatory evacuation of low-lying areas, pre-position rescue teams + prioritize underpasses and low-lying urban zones
Start evacuation: 23 hour(s) before impact
Evacuees: 105,000
Relief packs: 29,167
Rescue boats: 44




```
import json
import copy

CASE_BASE_FILE = "case_base.json"

#1. CASE BASE
#problem attributes
#disaster_type: "typhoon" | "flood" | "earthquake"
#area_type: "urban" | "rural" | "coastal"
#flood_risk: 1 (none), 5 (severe flooding)
#severity: 1 (weak), 5 (strong)
#population: number of people in the affected area
#lead_time_hours: hours of warning before impact
#solution attributes
#evacuation_strategy, evacuation_start_hours_before, evacuees,
#relief_packs, rescue_boats, medical_teams, notes
def seed_case_base():
    return [
        {
            "id": 1,
            "name": "Typhoon Ursula (2019)",
            "problem": {
                "disaster_type": "typhoon", "area_type": "coastal",
```

```
            "flood_risk": 3, "severity": 3,
            "population": 120000, "lead_time_hours": 48,
        },
        "solution": {
            "evacuation_strategy": "Pre-emptive evacuation of coastal barangays to school shelters",
            "evacuation_start_hours_before": 24,
            "evacuees": 30000, "relief_packs": 8000,
            "rescue_boats": 10, "medical_teams": 6,
        },
    },
    {
        "id": 2,
        "name": "Typhoon Odette (2021)",
        "problem": {
            "disaster_type": "typhoon", "area_type": "urban",
            "flood_risk": 4, "severity": 5,
            "population": 300000, "lead_time_hours": 36,
        },
        "solution": {
            "evacuation_strategy": "Mandatory evacuation of low-lying areas, pre-position rescue teams",
            "evacuation_start_hours_before": 30,
            "evacuees": 90000, "relief_packs": 25000,
            "rescue_boats": 30, "medical_teams": 15,
        },
    },
    {
        "id": 3,
        "name": "Typhoon Ondoy / Ketsana (2009)",
        "problem": {
            "disaster_type": "flood", "area_type": "urban",
            "flood_risk": 5, "severity": 4,
            "population": 500000, "lead_time_hours": 6,
        },
        "solution": {
            "evacuation_strategy": "Emergency rooftop and boat rescue, then evacuation to high-ground
centers",
            "evacuation_start_hours_before": 3,
            "evacuees": 150000, "relief_packs": 40000,
            "rescue_boats": 60, "medical_teams": 25,
        },
    },
    {
        "id": 4,
        "name": "Bohol Earthquake (2013)",
```

```
        "problem": {
            "disaster_type": "earthquake", "area_type": "rural",
            "flood_risk": 1, "severity": 4,
            "population": 80000, "lead_time_hours": 0,
        },
        "solution": {
            "evacuation_strategy": "Open-field evacuation camps, structural inspection of buildings",
            "evacuation_start_hours_before": 0,
            "evacuees": 40000, "relief_packs": 12000,
            "rescue_boats": 0, "medical_teams": 12,
        },
    },
    {
        "id": 5,
        "name": "Habagat Flooding (2012)",
        "problem": {
            "disaster_type": "flood", "area_type": "urban",
            "flood_risk": 4, "severity": 3,
            "population": 400000, "lead_time_hours": 12,
        },
        "solution": {
            "evacuation_strategy": "Early evacuation of riverside communities, activate pumping stations",
            "evacuation_start_hours_before": 8,
            "evacuees": 100000, "relief_packs": 30000,
            "rescue_boats": 40, "medical_teams": 18,
        },
    },
]

#2.SIMILARITY ASSESSMENT
WEIGHTS = {
    "disaster_type": 0.25,
    "area_type": 0.15,
    "flood_risk": 0.20,
    "severity": 0.15,
    "population": 0.15,
    "lead_time_hours": 0.10,
}

TYPE_SIMILARITY = {
    frozenset(["typhoon", "flood"]): 0.6,
    frozenset(["typhoon", "earthquake"]): 0.1,
    frozenset(["flood", "earthquake"]): 0.1,
}
```

```
def sim_disaster_type(a, b):
    if a == b:
        return 1.0
    return TYPE_SIMILARITY.get(frozenset([a, b]), 0.0)

def sim_area_type(a, b):
    if a == b:
        return 1.0
    #urban and coastal share dense settlements
    return 0.5 if {a, b} == {"urban", "coastal"} else 0.0

def sim_scale(a, b, max_range):
    """Numeric closeness on a fixed scale: 1.0 = identical, 0.0 = max apart."""
    return 1 - abs(a - b) / max_range

def sim_ratio(a, b):
    """Closeness for open-ended numbers such as population."""
    if max(a, b) == 0:
        return 1.0
    return min(a, b) / max(a, b)

def similarity(new_problem, old_problem):
    """Return (total_score, per-attribute scores) between two problems."""
    parts = {
        "disaster_type": sim_disaster_type(new_problem["disaster_type"], old_problem["disaster_type"]),
        "area_type": sim_area_type(new_problem["area_type"], old_problem["area_type"]),
        "flood_risk": sim_scale(new_problem["flood_risk"], old_problem["flood_risk"], 4),
        "severity": sim_scale(new_problem["severity"], old_problem["severity"], 4),
        "population": sim_ratio(new_problem["population"], old_problem["population"]),
        "lead_time_hours": sim_scale(
            min(new_problem["lead_time_hours"], 48),
            min(old_problem["lead_time_hours"], 48), 48),
    }
    total = sum(WEIGHTS[k] * parts[k] for k in parts)
    return total, parts

#3. CBR CYCLE FUNCTIONS
def retrieve(new_problem, case_base):
    """RETRIEVE: rank every stored case by similarity (highest first)."""
    ranked = []
    for case in case_base:
        score, parts = similarity(new_problem, case["problem"])
        ranked.append((score, case, parts))
```

```
    ranked.sort(key=lambda item: item[0], reverse=True)
    return ranked

def reuse_and_adapt(new_problem, best_case):
    """
    REUSE + ADAPT: start from the retrieved solution, then modify it so it
    fits the differences between the new and the old problem.
    Returns the adapted solution and a log explaining each adaptation.
    """
    old_p = best_case["problem"]
    sol = copy.deepcopy(best_case["solution"])
    log = []

    #a. Scale resource quantities by population size.
    scale = new_problem["population"] / old_p["population"]
    for key in ("evacuees", "relief_packs", "medical_teams"):
        before = sol[key]
        sol[key] = round(before * scale)
        log.append(f"{key}: {before:,} x population ratio {scale:.2f} -> {sol[key]:,}")

    #b. Rescue boats depend on population and flood risk
    boats_before = sol["rescue_boats"]
    flood_factor = new_problem["flood_risk"] / old_p["flood_risk"]
    sol["rescue_boats"] = round(boats_before * scale * flood_factor)
    log.append(f"rescue_boats: {boats_before} x pop {scale:.2f} x flood {flood_factor:.2f} "
               f"-> {sol['rescue_boats']}")

    #c. Higher severity than the old case, then evacuate a larger share
    if new_problem["severity"] > old_p["severity"]:
        extra = 1 + 0.10 * (new_problem["severity"] - old_p["severity"])
        before = sol["evacuees"]
        sol["evacuees"] = round(before * extra)
        log.append(f"evacuees raised by {int((extra - 1) * 100)}% (higher severity): "
                   f"{before:,} -> {sol['evacuees']:,}")

    #d. Lead time: if less warning than before, start evacuating earlier
    old_start = sol["evacuation_start_hours_before"]
    if new_problem["lead_time_hours"] < old_start:
        sol["evacuation_start_hours_before"] = max(new_problem["lead_time_hours"] - 1, 0)
        log.append(f"evacuation start cut from {old_start}h to "
                   f"{sol['evacuation_start_hours_before']}h before impact (shorter warning time)")
    else:
        log.append(f"evacuation start kept at {old_start}h before impact")
```

```
    #e. Area-specific note for the strategy
    if new_problem["area_type"] == "urban" and new_problem["flood_risk"] >= 4:
        sol["evacuation_strategy"] += " + prioritize underpasses and low-lying urban zones"
        log.append("strategy extended for urban flooding")

    return sol, log

def revise(solution, feedback):
    """
    REVISE: fix the proposed solution using feedback from a simulated
    deployment (e.g., what the LGU actually observed on the ground).
    """
    revised = copy.deepcopy(solution)
    log = []
    if feedback.get("shelter_shortfall_pct", 0) > 0:
        pct = feedback["shelter_shortfall_pct"]
        before = revised["evacuees"]
        revised["evacuees"] = round(before * (1 + pct / 100))
        log.append(f"Shelters ran short by {pct}% -> planned evacuees {before:,} -> {revised['evacuees']:,}")
    if feedback.get("boats_insufficient"):
        revised["rescue_boats"] += feedback["boats_insufficient"]
        log.append(f"{feedback['boats_insufficient']} more rescue boats needed -> {revised['rescue_boats']}")
    if not log:
        log.append("No corrections needed.")
    return revised, log

def retain(case_base, name, problem, solution):
    """RETAIN: store the newly solved case so the system can learn from it."""
    new_case = {
        "id": max(c["id"] for c in case_base) + 1,
        "name": name,
        "problem": problem,
        "solution": solution,
    }
    case_base.append(new_case)
    with open(CASE_BASE_FILE, "w") as f:
        json.dump(case_base, f, indent=2)
    return new_case

#4. DISPLAY HELPERS
LINE = "=" * 72

def show_problem(p):
    for k, v in p.items():
```

```
        print(f"   {k:<18}: {v:,}" if isinstance(v, int) and v > 999 else f"   {k:<18}: {v}")

def show_solution(s):
    print(f"Strategy: {s['evacuation_strategy']}")
    print(f"Start evacuation: {s['evacuation_start_hours_before']} hour(s) before impact")
    print(f"Evacuees: {s['evacuees']:,}")
    print(f"Relief packs: {s['relief_packs']:,}")
    print(f"Rescue boats: {s['rescue_boats']}")
    print(f"Medical teams: {s['medical_teams']}")

def show_case_base(case_base):
    print(f"   {'ID':<4}{'Case':<44}{'Type':<12}{'Area':<9}{'Pop.':>9}")
    print("   " + "-" * 76)
    for c in case_base:
        p = c["problem"]
        print(f"   {c['id']:<4}{c['name']:<44}{p['disaster_type']:<12}{p['area_type']:<9}{p['population']:>9,}")

#5. MAIN
def main():
    case_base = seed_case_base()

    print(LINE)
    print("DISASTER RESPONSE DSS - CASE-BASED REASONING")
    print(LINE)
    print(f"\nCase base loaded: {len(case_base)} past cases")
    show_case_base(case_base)

    new_problem = {
        "disaster_type": "typhoon",
        "area_type": "urban",
        "flood_risk": 5,
        "severity": 4,
        "population": 350000,
        "lead_time_hours": 24,
    }
    print("\n" + LINE)
    print("[INPUT] NEW PROBLEM: Typhoon with expected flooding in urban areas")
    print(LINE)
    show_problem(new_problem)

    #STEP 1
    print("\n" + LINE)
    print("[STEP 1 - RETRIEVE] Similarity ranking")
    print(LINE)
```

```
ranked = retrieve(new_problem, case_base)
for rank, (score, case, _) in enumerate(ranked, start=1):
    print(f"   #{rank}  {case['name']:<34} similarity = {score:.2%}")

best_score, best_case, best_parts = ranked[0]
second_score, second_case, _ = ranked[1]
print(f"\n   Most relevant case : {best_case['name']} ({best_score:.2%})")
print(f"   Runner-up         : {second_case['name']} ({second_score:.2%})")
print("\n   Attribute breakdown for the best match (weight x match):")
for attr, val in best_parts.items():
    print(f"      {attr:<18} {WEIGHTS[attr]:.2f} x {val:.2f} = {WEIGHTS[attr] * val:.3f}")
print("\n   Retrieved solution (before adaptation):")
show_solution(best_case["solution"])

#STEP 2
print("\n" + LINE)
print("[STEP 2 - REUSE & ADAPT] Fitting the old solution to the new problem")
print(LINE)
adapted, adapt_log = reuse_and_adapt(new_problem, best_case)
for line in adapt_log:
    print("   *", line)
print("\n   Adapted solution:")
show_solution(adapted)

#STEP 3
print("\n" + LINE)
print("[STEP 3 - REVISE] Applying simulated field feedback")
print(LINE)
feedback = {"shelter_shortfall_pct": 10, "boats_insufficient": 5}
print(f"   Feedback received: {feedback}")
final_solution, revise_log = revise(adapted, feedback)
for line in revise_log:
    print("   *", line)
print("\n   FINAL SOLUTION:")
show_solution(final_solution)

#STEP 4
print("\n" + LINE)
print("[STEP 4 - RETAIN] Learning from the new case")
print(LINE)
new_case = retain(case_base, "New Typhoon - Urban Flooding (Simulated)",
                  new_problem, final_solution)
print(f"   New case saved as ID {new_case['id']} -> '{new_case['name']}' (stored in {CASE_BASE_FILE})")
print(f"\n   Updated case base: {len(case_base)} cases")
```

```
    show_case_base(case_base)
    print("\n" + LINE)
    print("End of CBR cycle: Retrieve -> Reuse -> Revise -> Retain")
    print(LINE)

if __name__ == "__main__":
    main()
```